

LECTURE 5: DEBUGGING & EXCEPTIONS

Dr. Ehab Essa
Computer Science Department,
Mansoura University

HOW TO AVOID DEBUGGING

FIRST DEFENSE: MAKE BUGS IMPOSSIBLE

One way that we've already talked about is **static checking**. Static checking **eliminates many bugs by catching them at compile time.**

Dynamic checking:

- For example, Java makes array overflow bugs impossible by catching them dynamically. If you try to use an index outside the bounds of an array or a List, then Java automatically produces an error.

Immutability is another design principle that prevents bugs. An immutable type is a type whose values can never change once they have been created.

- **String is an immutable type.** Strings can be passed around and shared without fear that they will be modified by other code.

Java also gives us **immutable references**: variables declared with the **keyword final**.

- It's good practice to use final for declaring the parameters of a method and as many local variables as possible.
- Like the type of the variable, these declarations are important documentation, useful to the reader of the code and statically checked by the compiler.

IMMUTABLE TYPE

```
public class TestStringImmutability {  
    public static void main(String[] args) {  
        String originalString = "hello";  
        String modifiedString = modifyString(originalString);  
  
        System.out.println("Original: " + originalString); // This will print "Original: hello"  
        System.out.println("Modified: " + modifiedString); // This will print "Modified: hello world"  
    }  
  
    public static String modifyString(String input) {  
        input = input + " world"; // A new String object is created here  
        return input; // Return the new String object  
    }  
}
```

- **originalString** remains unchanged even after being passed to the modifyString method because strings in Java are immutable.
- What happened if you use StringBuilder instead of String?

SECOND DEFENSE: LOCALIZE BUGS

If we can't prevent bugs, we can **try to localize them** to a small part of the program. When localized to a single method or small module, bugs may be found simply by studying the program text.

```
/**
 * @param x  requires x >= 0
 * @return approximation to square root of x
 */
public double sqrt(double x) { ... }
```

Now suppose somebody *calls sqrt with a negative argument*.

Since the bad call indicates a bug in the caller, the most useful behavior would point out the bug as early as possible. We do this by throwing an AssertionError **exception** that tests the precondition.

```
/**
 * @param x  requires x >= 0
 * @return approximation to square root of x
 */
public double sqrt(double x) {
    if (! (x >= 0)) throw new AssertionError();
    ...
}
```

ASSERTIONS

Checking preconditions is an example of **defensive programming**. The core idea is to anticipate potential errors and handle them proactively

Defensive programming offers a way to mitigate the effects of bugs even if you don't know where they are.

an **assertion** is a statement that a programmer believes to be true at a specific point in the code. It's used during debugging to catch errors in the code logic. Assertions have the added benefit of **documenting an assumption** about the state of the program at that point.

assert x >= 0; it says “at this point, it should always be true that $x \geq 0$.” Unlike a comment, however, an assertion is executable code that enforces the assumption at runtime.

The simplest form of the assert statement takes a boolean expression, exactly as shown above, and **throws AssertionError** if the boolean expression evaluates to false

ASSERTIONS

An **assert statement** may also return a **description expression**, which is usually a string, but may also be a primitive type or a reference to an object. The description is printed in an error message when the assertion fails, so it can be used to provide additional details to the programmer about the cause of the failure.

```
assert (x >= 0) : "x is " + x;
```

```
x is -1
```

A serious problem with Java assertions is that assertions are *off by default*.

Java's designers did this because checking assertions can **sometimes be costly to performance**. You should be willing (eager!) to pay this cost during testing, since it makes debugging much easier, but not after the program is released to users.

WHAT TO ASSERT

Method argument requirements, like we saw for sqrt.

Method return value requirements. This kind of assertion is sometimes called a self check. For example, the sqrt method might square its result to check whether it is reasonably close to x

Covering all cases. If a conditional statement or switch does not cover all the possible cases, it is good practice to use an assertion to block the illegal cases

When should you write runtime **assertions**?

- As you write the code, not after the fact. When you're writing the code, you have the invariants in mind.
- If you postpone writing assertions, you're less likely to do it, and you're liable to omit some important invariants.

```
public double sqrt(double x) {  
    assert x >= 0;  
    double r;  
    ... // compute result r  
    assert Math.abs(r*r - x) < .0001;  
    return r;  
}
```

```
switch (vowel) {  
    case 'a':  
    case 'e':  
    case 'i':  
    case 'o':  
    case 'u': return "A";  
    default: assert false;  
}
```

WHAT NOT TO ASSERT

Runtime assertions are not free. They can clutter the code, so they must be used judiciously. **Avoid trivial assertions**, just as you would avoid uninformative comments.

```
// don't do this:  
x = y + 1;  
assert x == y+1;
```

Never use assertions to test conditions that are external to your program, such as the existence of files, the availability of the network, or the correctness of input typed by a human user. **External failures are not bugs**, and there is no change you can make to your program in advance that will prevent them from happening. External failures should be handled using exceptions instead.

Many assertion mechanisms are designed so that **assertions are executed only during testing and debugging**, and **turned off** when the program is **released** to users. **Since assertions may be disabled, the correctness of your program should never depend on whether or not the assertion expressions are executed.** In particular, asserted expressions should not have *side-effects*.

```
// don't do this:  
assert list.remove(x);
```

```
boolean found = list.remove(x);  
assert found;
```


MODULARITY & ENCAPSULATION

You can also localize bugs by better software design.

Modularity. means dividing up a system into components, or modules, each of which can be designed, implemented, tested, reasoned about, and reused separately from the rest of the system. The opposite of a modular system is **a monolithic system** – big and with all of its pieces tangled up and dependent on each other.

Encapsulation. means bundling data and corresponding methods (behavior) together into a single unit and building walls around a module so that the module is **responsible for its own internal behavior**, and bugs in other parts of the system can't damage its integrity.

One kind of encapsulation is **access control**, using **public and private** to control the visibility and accessibility of your variables and methods.

Keeping things private as much as possible, especially for variables, provides encapsulation, since it limits the code that could inadvertently cause bugs.

MODULARITY & ENCAPSULATION

Another kind of encapsulation comes from **variable scope**. By properly scoping variables, you prevent them from being **accessible where they're not needed**.

The **scope of a variable** is the **portion of the program text over which that variable is defined, in the sense that expressions and statements can refer to the variable**.

A method parameter's scope is the body of the method.

A local variable's scope extends from its declaration to the next closing curly brace of that block.

Keeping variable **scopes as small as possible** makes it much easier to reason about where a bug might be in the program. For example, suppose you have a loop like this:

```
public static int i;  
...  
for (i = 0; i < 100; ++i) {  
    ...  
    doSomeThings();  
    ...  
}
```

and you've discovered that this loop keeps running forever – i never reaches 100. Somewhere, somebody is changing i

VARIABLE SCOPE

Minimizing the scope of variables is a powerful practice for bug localization. Here are a few rules that are good for Java:

- **Always declare a loop variable in the for-loop initializer.** So rather than declaring it before the loop:

```
int i;  
for (i = 0; i < 100; ++i) {
```

```
for (int i = 0; i < 100; ++i) {
```

- **Declare a variable only when you first need it, and in the innermost curly-brace block that you can.** Don't declare all your variables at the start of the function – it makes their scopes unnecessarily large. But note that in languages without static type declarations, like Python and Javascript, the scope of a variable is normally the entire function anyway, so you can't restrict the scope of a variable with curly braces, alas.
- **Avoid global variables.** Very bad idea, especially as programs get large. Global variables are often used as a shortcut to provide a parameter to several parts of your program. **It's better to just pass the parameter into the code that needs it, rather than putting it in global space where it can inadvertently be reassigned.**

REPRODUCE THE BUG

Sometimes you have no choice but to debug – particularly when the bug is found only when you plug the whole system together, or reported by a user after the system is deployed, in which case it may be hard to localize it to a particular module. For those situations, we can suggest a **systematic strategy** for more effective debugging.

Start **by finding a small**, repeatable test case that produces the failure.

If the bug was found by regression testing, then you're in luck; you already have a failing test case in your test suite.

If the bug was reported by a user, it may take some effort to reproduce the bug. For graphical user interfaces and multithreaded programs, a bug may be hard to reproduce consistently if it depends on timing of events or thread execution.

UNDERSTAND THE LOCATION AND CAUSE OF THE BUG

To localize the bug and its cause, you can use the scientific method:

Study the data. Look at the test input that causes the bug, and the incorrect results, failed assertions, and stack traces that result from it.

Hypothesize. Propose a hypothesis, consistent with all the data, about where the bug might be, or where it cannot be. It's good to make this hypothesis general at first.

Experiment. Devise an experiment that tests your hypothesis. It's good to make the experiment an observation at first – a probe that collects information but disturbs the system as little as possible. Run a different test case. Insert a print statement or assertion in the running program, to check something about its internal state. Set a breakpoint using a debugger, then single-step through the code and look at variable and object values.

Repeat. Add the data you collected from your experiment to what you knew before, and make a fresh hypothesis. Hopefully you have ruled out some possibilities and narrowed the set of possible locations and reasons for the bug.

OTHER TIPS

Bug localization by binary search. Debugging is a search process, and you can sometimes use **binary search** to speed up the process.

Prioritize your hypotheses. When making your hypothesis, you may want to keep in mind that different parts of the system have different likelihoods of failure. For example, old, well-tested code is probably more trustworthy than recently-added code. Java library code is probably more trustworthy than yours. The Java compiler and runtime, operating system platform, and hardware are increasingly more trustworthy, because they are more tried and tested. You should trust these lower levels until you've found good reason not to.

Swap components. If you have another implementation of a module that satisfies the same interface, and you suspect the module, then one experiment you can do is to try swapping in the alternative.

Make sure your source code and object code are up to date. Pull the latest version from the repository, and delete all your binary files and recompile everything (in Eclipse, this is done by Project → Clean).

Get help. It often helps to explain your problem to someone else, even if the person you're talking to has no idea what you're talking about.

Sleep on it. If you're too tired, you won't be an effective debugger. Trade latency for efficiency.

FIX THE BUG

Once you've found the bug and understand its cause, the third step is to devise a fix for it.

Avoid the temptation to slap a patch on it and move on. Ask yourself whether the bug was a coding error, like a misspelled variable or interchanged method parameters, or a design error, like an underspecified or insufficient interface. Design errors may suggest that you step back and revisit your design, or at the very least consider all the other clients of the failing interface to see if they suffer from the bug too.

Think also whether the bug has any relatives. If I just found a **divide-by-zero error here**, did I do that anywhere else in the code? Try to make the code safe from future bugs like this. Also consider what effects your fix will have. Will it break any other code?

Finally, after you have applied your fix, **add the bug's test case to your regression test suite**, and run all the tests to assure yourself that (a) the bug is fixed, and (b) no new bugs have been introduced.

EXCEPTIONS

EXCEPTIONS

An **exception** is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

When an error occurs within a method, the method creates an object called an **exception object** and hands it off to the runtime system. The object contains information about the error. This process called **throwing an exception**.

After a method throws an exception, the runtime system attempts to find something to handle it. It created an ordered list of methods that had been called to get to the method where the error occurred. The list of methods is known as the **call stack**.

The runtime system searches the call stack for a method that contains a block of code that can handle the exception. This block of code is called **an exception handler**.

The core advantage of exception handling is to maintain the normal flow of the application

EXCEPTIONS

Exceptions for signaling bugs

These exceptions generally indicate bugs in your code, and the information displayed by Java when the exception is thrown can help you find and fix the bug:

ArrayIndexOutOfBoundsException (thrown when an array index `foo[i]` is outside the valid range for the array `foo`)

NullPointerException (thrown when trying to call a method on a null object reference).

ArithmeticException, thrown for arithmetic errors like integer division by zero.

NumberFormatException, thrown by methods like `Integer.parseInt` if you pass in a string that cannot be parsed into an integer.

EXCEPTIONS

Exceptions for special results

They can be used to improve the structure of code that involves procedures with special results.

An unfortunately common way to handle special results is to return special values.

- Lookup operations in the Java library are often designed like this: you get an index of -1 when expecting a positive integer, or a null reference when expecting an object.

This approach has two problems.

- First, it's tedious to check the return value. Second, it's easy to forget to do it.
- Also, it's not always easy to find a 'special value'
- Suppose we have a BirthdayBook class with a lookup method

```
class BirthdayBook {  
    LocalDate lookup(String name) { ... }  
}
```

- Bad programmers have been doing this for decades; they would return 9/9/99, for example, since it was obvious that no program written in 1960 would still be running at the end of the century.

EXAMPLE 1

Let's define the method *lookup* in BirthdayBook class:

NotFoundException is a **checked exception**, and that's why the signature ends throws NotFoundException

The caller handles the exception with a catch clause. For example:

```
LocalDate lookup(String name) throws NotFoundException {  
    ...  
    if ( ...not found... )  
        throw new NotFoundException();  
    ...  
}
```

```
BirthdayBook birthdays = ...  
try {  
    LocalDate birthdate = birthdays.lookup("Alyssa");  
    // we know Alyssa's birthday  
} catch (NotFoundException nfe) {  
    // her birthday was not in the birthday book  
}
```

CHECKED AND UNCHECKED EXCEPTIONS

Checked exceptions are the exceptions that are checked at compile time. To signal special results. These are exceptional conditions that a well-written application should anticipate and recover from.

Checked exceptions *are subject* to the **Specify or Catch** Requirement:

- If a user defined method might throw a checked exception, the possibility must be **declared in its signature using throws keyword**.
- If a method calls another method that may throw a checked exception, it must either **handle it (try ... catch)** , or declare the exception itself using **throws** keyword, since if it isn't caught locally it will be propagated up to callers.

EXAMPLE

What kind of exceptions are in this code?

```
import java.io.*;
class Example {
    public static void main(String args[])
    {
        FileInputStream fis = null;
        fis = new FileInputStream("B:/myfile.txt");
        int k;
        while(( k = fis.read() ) != -1)
        {
            System.out.print((char)k);
        }

        fis.close();
    }
}
```

Exception in thread "main" java.lang.Error: Unresolved compilation problems:
Unhandléd exception type FileNotFoundException
Unhandléd exception type IOException
Unhandléd exception type IOException

METHOD 1: DECLARE THE EXCEPTION USING THROWS KEYWORD.

You may be thinking that our code is throwing `FileNotFoundException` and `IOException` both then why we are declaring the `IOException` alone.

The reason is that `IOException` is a parent class of `FileNotFoundException` so it by default covers that.

If you want you can declare them like this

```
public static void main(String args[])  
throws IOException,  
FileNotFoundException.
```

```
import java.io.*;  
class Example {  
    public static void main(String args[]) throws IOException  
    {  
        FileInputStream fis = null;  
        fis = new FileInputStream("B:/myfile.txt");  
        int k;  
  
        while(( k = fis.read() ) != -1)  
        {  
            System.out.print((char)k);  
        }  
        fis.close();  
    }  
}
```

METHOD 1: DECLARE THE EXCEPTION USING THROWS KEYWORD.

Will this code successfully compile?

The **main** method, which calls `readFile`, does not handle or declare this exception. This will cause a compilation error because unhandled checked exceptions are not allowed in Java.

```
import java.io.*;
class Example {

    public static void readFile(String filePath) throws IOException
    {
        FileInputStream fis = null;
        fis = new FileInputStream(filePath);
        int k;
        while ((k = fis.read()) != -1) {
            System.out.print((char) k);
        }
        fis.close();
    }

    public static void main(String[] args) {
        readFile("B:/myfile.txt");
    }
}
```


METHOD 2: HANDLE THEM USING TRY-CATCH BLOCKS.

The previous method is not the best exception handling practice.

You should give meaningful message for each exception type so that it would be easy for someone to understand the error.

```
import java.io.*;
class Example {
    public static void main(String args[])
    {
        FileInputStream fis = null;
        try{
            fis = new FileInputStream("B:/myfile.txt");
        }catch(FileNotFoundException fnfe){
            System.out.println("The specified file is not " +
                               "present at the given path");
        }
        int k;
        try{
            while(( k = fis.read() ) != -1)
            {
                System.out.print((char)k);
            }
            fis.close();
        }catch(IOException ioe){
            System.out.println("I/O error occurred: "+ioe);
        }
    }
}
```

UNCHECKED EXCEPTIONS

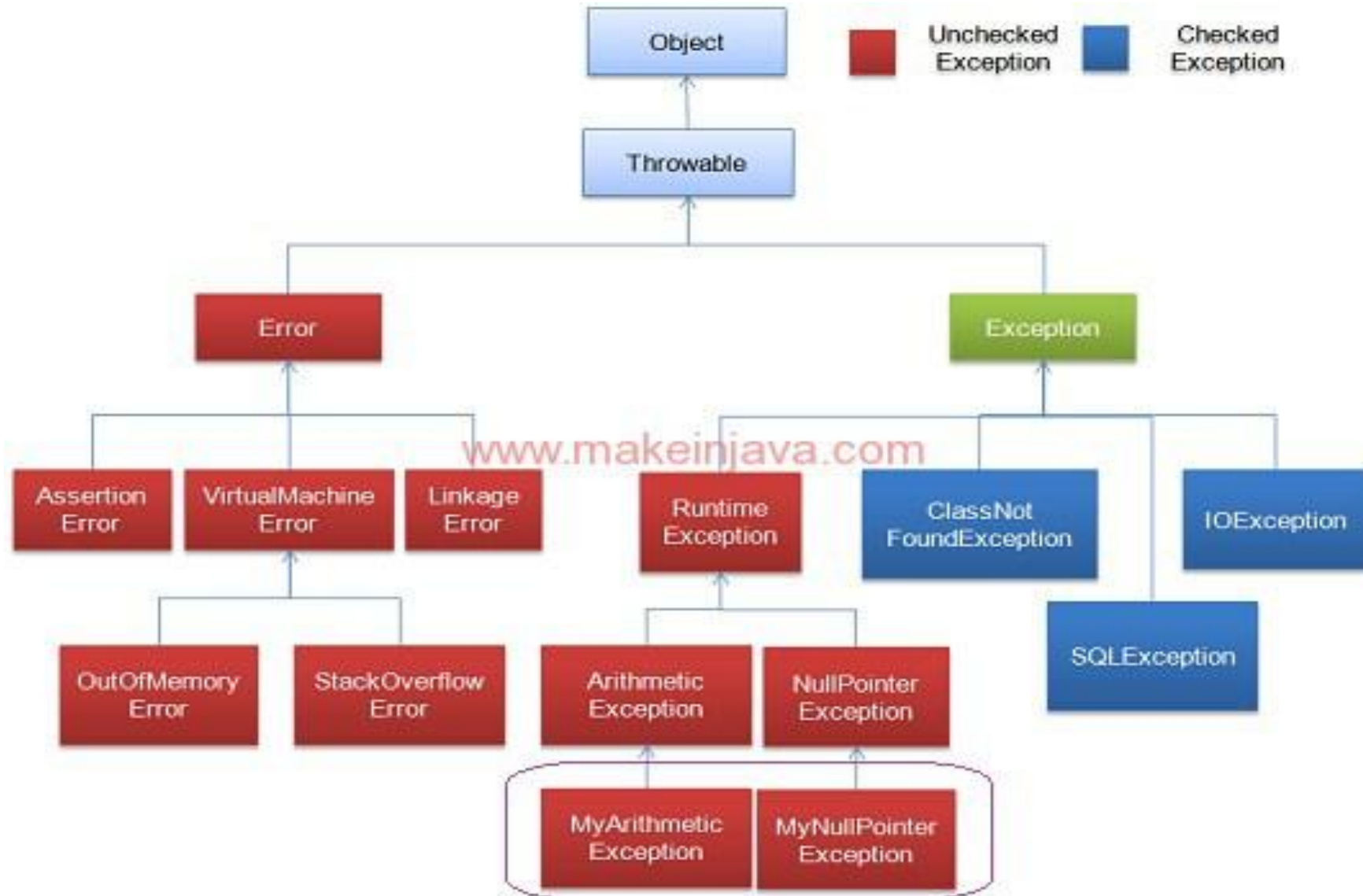
Unchecked exceptions, are used to **signal bugs**. It is not forced by the compiler to either handle or specify the exception. It is up to the programmers to specify or catch the exceptions.

- The code will compile even if you don't put the operation that may throw such an exception inside a try-catch block.

Error are **exceptional conditions** that are external to the application, and that the application usually cannot anticipate or recover from. For example, **OutOfMemoryError**: Thrown when the Java Virtual Machine cannot allocate an object because it's out of memory and no more memory could be made available by the garbage collector.

Runtime exception are exceptional conditions that are internal to the application, and that the application usually cannot anticipate or recover from. These usually indicate programming bugs, such as logic errors or improper use of an API. For example, consider the application described previously that passes a file name to the constructor for `FileReader`. If a logic error causes a null to be passed to the constructor, the constructor will throw `NullPointerException`.

CHECKED AND UNCHECKED EXCEPTIONS



RUNTIME EXCEPTION

1) Scenario where ArithmeticException occurs

```
int a=50/0;//ArithmeticException
```

2) Scenario where NullPointerException occurs

If we have null value in any variable, performing any operation by the variable occurs an NullPointerException.

```
String s=null;  
System.out.println(s.length());//NullPointerException
```

3) Scenario where NumberFormatException occurs

The wrong formatting of any value, may occur NumberFormatException. Suppose I have a string variable that have characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";  
int i=Integer.parseInt(s);//NumberFormatException
```

4) Scenario where ArrayIndexOutOfBoundsException occurs

If you are inserting any value in the wrong index, it would result ArrayIndexOutOfBoundsException as shown below:

```
int a[]=new int[5];  
a[10]=50; //ArrayIndexOutOfBoundsException
```

THROW KEYWORD

We can define our own set of conditions or rules and throw an exception explicitly using **throw** keyword.

For example, we can throw `ArithmeticException` when we divide number by 5, or any other numbers, what we need to do is just set the condition and throw any exception using throw keyword.

Throw keyword can also be used for throwing custom exceptions.

Syntax of throw keyword:

```
throw new exception_class("error message");
```

For example:

```
throw new ArithmeticException("dividing a number by 5 is not allowed in this program");
```

EXAMPLE

we need to only register the students when their age is less than 12 and weight is less than 40.

If any of the condition is not met then the user should get an `ArithmeticException`

```
/* In this program we are checking the Student age
 * if the student age<12 and weight <40 then our program
 * should return that the student is not eligible for registration.
 */
public class ThrowExample {
    static void checkEligibility(int stuage, int stuweight){
        if(stuage<12 && stuweight<40) {
            throw new ArithmeticException("Student is not eligible for registration");
        }
        else {
            System.out.println("Student Entry is Valid!!");
        }
    }

    public static void main(String args[]){
        System.out.println("Welcome to the Registration process!!");
        checkEligibility(10, 39);
        System.out.println("Have a nice day..");
    }
}
```

```
Welcome to the Registration process!!Exception in thread "main"
java.lang.ArithmeticException: Student is not eligible for registration
at beginnersbook.com.ThrowExample.checkEligibility(ThrowExample.java:9)
at beginnersbook.com.ThrowExample.main(ThrowExample.java:18)
```

THROWS CLAUSE

Throws keyword is used to declare and handle checked exceptions. It force the user to handle the exception when you call a method used throws.

```
import java.io.IOException;

/* In this program we are checking the Student age
 * if the student age<12 and weight <40 then our program
 * should return that the student is not eligible for registration.
 */
public class helloworld {
    static void checkEligibility(int stuage, int stuweight) throws IOException
    {
        if(stuage<12 && stuweight<40) {
            throw new IOException("Student is not eligible for registration");
        }
        else {
            System.out.println("Student Entry is Valid!!");
        }
    }

    public static void main(String args[]) throws IOException
    {
        System.out.println("Welcome to the Registration process!!");
        checkEligibility(10, 39);
        System.out.println("Have a nice day..");
    }
}
```

```
public class helloworld {
    static void checkEligibility(int stuage, int stuweight) throws IOException
    {
        if(stuage<12 && stuweight<40) {
            throw new IOException("Student is not eligible for registration");
        }
        else {
            System.out.println("Student Entry is Valid!!");
        }
    }

    public static void main(String args[])
    {
        System.out.println("Welcome to the Registration process!!");
        try
        {
            checkEligibility(10, 39);
        }
        catch(IOException e)
        {
            System.out.println(e);
        }

        System.out.println("Have a nice day..");
    }
}
```

JAVA CUSTOM EXCEPTION

By the help of custom exception, you can have your own exception and message.

```
class InvalidAgeException extends Exception{  
    InvalidAgeException(String s){  
        super(s);  
    }  
}
```

```
class TestCustomException1{  
  
    static void validate(int age)throws InvalidAgeException{  
        if(age<18)  
            throw new InvalidAgeException("not valid");  
        else  
            System.out.println("welcome to vote");  
    }  
  
    public static void main(String args[]){  
        try{  
            validate(13);  
        }catch(Exception m){System.out.println("Exception occured: "+m);}  
  
        System.out.println("rest of the code...");  
    }  
}
```


EXCEPTION DESIGN CONSIDERATIONS

Aside from the performance penalty, exceptions are a pain to use, in both method design and method use.

You should use an unchecked exception only to signal an unexpected failure (i.e. a bug), or if you expect that clients will usually write code that ensures the exception will not happen, because there is a **convenient** and **inexpensive** way to avoid the exception;

Otherwise you should use a checked exception.

Here are some examples:

- `Queue.pop()` throws an unchecked `EmptyQueueException` when the queue is empty, because it's reasonable to expect the caller to avoid this with a call like `Queue.size()` or `Queue.isEmpty()`.
- `Url.getWebPage()` throws a checked `IOException` when it can't retrieve the web page, because it's not easy for the caller to prevent this.
- `int integerSquareRoot(int x)` throws a checked `NotPerfectSquareException` when `x` has no integral square root, because testing whether `x` is a perfect square is just as hard as finding the actual square root, so it's not reasonable to expect the caller to prevent it.

ABUSE OF EXCEPTIONS

Here's an example from Effective Java by Joshua Bloch (Item 57 in the 2nd edition).

```
try {  
    int i = 0;  
    while (true)  
        a[i++].f();  
} catch (ArrayIndexOutOfBoundsException e) { }
```

It is supposed to be equivalent to:

```
for (int i = 0; i < a.length; i++) {  
    a[i].f();  
}
```